Setup & Runbook

Everything here is written for the actual target machine: Windows 11, PowerShell, Python 3.12 alongside 3.15, Node 24, PostgreSQL 18 already running, no Docker.

Step 0 — Initialise the repository

`c:\ocr` is not a git repository yet. Do this **before** the first commit, or every Alembic revision and config file will churn line endings forever.

```
cd C:\ocr
git init

# Must exist before the first `git add`.
@'
* text=auto eol=lf
*.png binary
*.pdf binary
'@ | Out-File -Encoding utf8 .gitattributes

@'
# Python
server/.venv/
server/venv/
__pycache__/
*.py[cod]
.pytest_cache/
.mypy_cache/
.ruff_cache/

# Secrets & data
.env
.env.local
server/storage/

# Node
client/node_modules/
client/.next/
client/out/

# Docs build artifacts
docs/pdf/*.html
.docvenv/
'@ | Out-File -Encoding utf8 .gitignore
```

Step 1 — Recreate the Python environment

The existing `server\venv` is Python **3.15.0**. `asyncpg`, `pydantic-core`, `greenlet` and `rapidfuzz` publish no cp315 wheels, so pip falls back to building from source and fails without MSVC Build Tools. Replace it with 3.12, which is already installed.

```
cd C:\ocr\server

# Confirm 3.12 is present.
py -0p          # expect: -V:3.12   ...\Programs\Python\Python312\python.exe

Remove-Item -Recurse -Force .\venv
py -3.12 -m venv .venv

.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
```

This is the riskiest step in the whole setup, because it validates every pinned version at once. Watch the output: you should see only `Downloading ...whl` lines. Any `Building wheel for ...` means a package had no matching wheel and is compiling from source.

```
# Verify: every import resolves, and nothing was built from source.
.\.venv\Scripts\python.exe -c "import fastapi, pydantic, sqlalchemy, asyncpg, alembic, rapidfuzz, jwt,
argon2, structlog; print('backend imports OK')"
.\.venv\Scripts\python.exe -c "from mistralai.client import Mistral; print('mistral v2 import OK')"
```

That second check matters: if `from mistralai.client import Mistral` fails but `from mistralai import Mistral` works, you have v1 installed and every code sample in document 03 will need the older import path.

Step 2 — Create the database

PostgreSQL 18 is already running as a service.

```
# Adjust the path if your PG install differs.
$psql = "C:\Program Files\PostgreSQL\18\bin\psql.exe"

& $psql -U postgres -c "CREATE DATABASE ap_automation;"
& $psql -U postgres -d ap_automation -c "CREATE EXTENSION IF NOT EXISTS pgcrypto;"
& $psql -U postgres -d ap_automation -c "SELECT version();"
```

Step 3 — Configure secrets

```
cd C:\ocr\server
Copy-Item .env.example .env
```

Generate the two keys:

```
.\.venv\Scripts\python.exe -c "import secrets; print('SECRET_KEY=' + secrets.token_urlsafe(48))"
.\.venv\Scripts\python.exe -c "from cryptography.fernet import Fernet; print('CREDENTIAL_ENCRYPTION_KEY=' +
Fernet.generate_key().decode())"
```

Paste both into `.env`, along with `MISTRAL_API_KEY` and your Odoo details.

> **Watch the BOM.** PowerShell's `>` and `Out-File` write UTF-8 *with* a byte-order mark. If the BOM lands at the start of `.env`, it becomes part of the first variable's name and that setting silently disappears. `env_file_encoding="utf-8-sig"` in `config.py` handles it, but if you hand-edit the file, save it as UTF-8 without BOM.

Verify settings load:

```
.\.venv\Scripts\python.exe -c "from app.core.config import settings; print(settings.PROJECT_NAME,
settings.ENVIRONMENT); print('DSN ok:', settings.sqlalchemy_dsn.split('@')[-1])"
```

Step 4 — Run the migrations

```
cd C:\ocr\server
.\.venv\Scripts\alembic.exe revision --autogenerate -m "init schema"
```

Open the generated file in `alembic/versions/` and check three things before applying it:

1. `op.execute("CREATE EXTENSION IF NOT EXISTS pgcrypto")` is the first statement.
2. All five enum types are created: `odoo_connection_status`, `user_role`, `alias_source`, `invoice_status`, `match_method`.
3. The `CheckConstraint`s on `vendor_knowledge_base` survived — Alembic is inconsistent about picking these up.

```
.\.venv\Scripts\alembic.exe upgrade head
```

Verify:

```
-- psql -U postgres -d ap_automation
\dt                                     -- 4 tables + alembic_version
\dT                                     -- 5 enum types
\d match_history                        -- 6 indexes, all leading with organization_id
SELECT enum_range(NULL::invoice_status);   -- 9 values
```

Step 5 — Start the backend

```
cd C:\ocr\server
$env:PYTHONUTF8 = "1"
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

`PYTHONUTF8=1` is not optional in practice. Vendor names contain `é`, `ü`, `ł`, and structlog's console renderer raises `UnicodeEncodeError` on a cp1252 terminal without it.

```
# In a second terminal:
Invoke-RestMethod http://127.0.0.1:8000/health        # {"status":"ok","env":"local"}
Invoke-RestMethod http://127.0.0.1:8000/health/ready  # {"status":"ready"}
```

`/health/ready` performing a real DB round trip is what makes it meaningful — if it returns `ready`, the async DSN, the driver and the credentials are all confirmed working.

Interactive docs (only when `DEBUG=true`): http://127.0.0.1:8000/docs

Step 6 — Connect Odoo

In Odoo, create an integration user with **Purchase → User** and **Invoicing → Billing** access rights, then generate an API key under *Preferences → Account Security → New API Key*.

```powershell
# Register the first organization + owner.
$body = @{
  email = "you@example.com"
  password = "a-real-password"
  full_name = "Your Name"
  organization_name = "Your Company"
} | ConvertTo-Json

$auth = Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/api/v1/auth/register `
  -ContentType "application/json" -Body $body
$headers = @{ Authorization = "Bearer $($auth.access_token)" }

# Save the Odoo credentials — this immediately tests the connection.
$odoo = @{
  url = "https://yourco.odoo.com"
  db = "yourco"
  username = "bot@yourco.com"
  api_key = "..."
} | ConvertTo-Json

Invoke-RestMethod -Method Put -Uri http://127.0.0.1:8000/api/v1/organization/odoo `
  -Headers $headers -ContentType "application/json" -Body $odoo
```

Expect `odoo_status: "ok"`. Then confirm POs come back **with their lines populated** — this is what proves the batched line read works, and it is the highest-uncertainty integration in the system:

```powershell
$pos = Invoke-RestMethod -Uri "http://127.0.0.1:8000/api/v1/odoo/purchase-orders?limit=5" -Headers $headers
$pos | Select-Object name, partner_name, amount_total, @{n='lines';e={$_.order_line.Count}}
```

If `lines` is 0 for every PO, the batched `purchase.order.line` read is failing silently — fix that before going any further, because the line-item component of the score will be permanently inapplicable.

Odoo troubleshooting

Symptom	Cause	Fix
`odoo_auth_failed`, "returned no uid"	Wrong database name — the most common error	The db is the subdomain for odoo.sh, visible at `/web/database/selector`
`AccessError` on `action_create_invoice`	Integration user lacks Billing rights	Add *Invoicing → Billing* to the user
`odoo_unavailable`	URL includes a path, or a firewall blocks it	Use the bare origin, no trailing path
POs return but `order_line` is empty	Batched read failing	Check `PO_LINE_FIELDS` matches your Odoo version's field names

Step 7 — Prove OCR

Upload five real invoices with matching disabled, so an OCR problem is not confused with a matching problem:

```powershell
curl.exe -X POST "http://127.0.0.1:8000/api/v1/invoices/upload?auto_match=false" `
  -H "Authorization: Bearer $($auth.access_token)" `
  -F "file=@C:\path\to\invoice.pdf"
```

Every one should come back with a non-null `vendor_name` and `total_amount`. **Save these JSON responses into `tests/fixtures/`** — they become the input to the matching engine's unit tests, which is the only part of the system you can then iterate on with no network access at all.

Check `vendor_name` carefully on each. If it is the *customer* rather than the supplier, the extraction prompt needs strengthening — that is rule 1 in `_ANNOTATION_PROMPT` and the single most common OCR failure on invoice templates where the customer's name is more prominent.

Step 8 — Tune the matching engine

```
.\.venv\Scripts\python.exe -m pytest tests/unit/test_matching_engine.py -v
```

Assert score **bands** (`high` / `medium` / `low`), never exact floats. Float assertions mean every weight adjustment breaks the entire suite, and a suite that breaks on every legitimate change gets deleted rather than maintained.

Add `asyncio_mode = "auto"` and `asyncio_default_fixture_loop_scope = "session"` to `pyproject.toml`. The `event_loop` fixture was removed in pytest-asyncio 1.x; use `@pytest_asyncio.fixture` for async fixtures.

Step 9 — Close the loop

```
$confirm = @{
  odoo_po_id = 42
  corrections = @()
  learn_alias = $true
  push_action = "create_bill"
  post_bill = $false
} | ConvertTo-Json

Invoke-RestMethod -Method Post `
  -Uri "http://127.0.0.1:8000/api/v1/invoices/$invoiceId/confirm" `
  -Headers $headers -ContentType "application/json" -Body $confirm
```

Then verify all four effects:

1. The row is `status: "pushed"` with an `odoo_bill_id`.
2. In Odoo, a **draft** `account.move` exists, with `ref` set to the OCR'd invoice number. It must be draft — if it is posted, `post_bill` leaked through as true.
3. `GET /vendors/kb` shows a new alias row for this vendor.
4. **Upload another invoice from the same vendor** — the response should now carry `match_method: "kb_alias"` and a materially higher score. That is the learning loop working, and it is the whole point of the system.

Step 10 — Frontend

```
cd C:\ocr\client
npx create-next-app@16.3.0 . --ts --tailwind --eslint --app --src-dir --import-alias "@/*"
# Then replace package.json with the pinned version from document 05.
npm install
npx shadcn@latest init -t next
npx shadcn@latest add alert alert-dialog avatar badge breadcrumb button card checkbox command dialog
dropdown-menu form input label pagination popover progress resizable scroll-area select separator sheet
skeleton sonner switch table tabs textarea tooltip

Copy-Item .env.example .env.local
npm run dev
```

```
npm run build ; npm run lint ; npm run typecheck   # all must pass clean
```

Confirm the `pdfjs-dist` pin held — this is the check that catches the single most likely frontend failure before you hit it at runtime:

```
npm ls pdfjs-dist
# Expect exactly one entry at 5.4.296. Two entries, or any 6.x, means the
# override did not take and the PDF viewer will throw
# "The API version does not match the Worker version".
```

Full verification checklist

Run in order. Each step depends on the previous one passing.

#	Check	Pass criteria
1	`pip install -r requirements-dev.txt`	Completes with no source builds
2	`alembic upgrade head`	4 tables, 5 enum types created
3	`/health` and `/health/ready`	Both 200
4	Register → login → `/auth/me`	Returns the user with embedded organization
5	`PUT /organization/odoo`	`odoo_status: "ok"`
6	`GET /odoo/purchase-orders`	POs returned **with lines populated**
7	Upload 5 invoices, `auto_match=false`	Non-null `vendor_name` and `total_amount` on each
8	`pytest tests/unit/`	Green; asserts bands, not floats
9	`POST /invoices/{id}/confirm`	Row `pushed`; **draft** bill in Odoo; alias learned
10	Re-upload same vendor	`match_method: "kb_alias"`, higher score
11	`npm run build && lint && typecheck`	All clean
12	`npm ls pdfjs-dist`	Exactly one entry, 5.4.296
13	Frontend end to end	Login sets httpOnly cookie → signed-out `/dashboard` redirects → upload → PDF renders in left pane → confirm → row shows `pushed`

Windows-specific gotchas

Collected in one place, because each of these costs an hour if you meet it cold.

1. **No uvloop.** `uvicorn[standard]` marks it `sys_platform != 'win32'`, so pip skips it silently and the stdlib asyncio loop is used. Never add `uvloop` explicitly — it has no Windows wheel. Expect 20–30% lower raw throughput than Linux, which is irrelevant here since the workload is I/O-bound on Mistral and Odoo.
2. **asyncpg on the Proactor loop.** The Windows default works. On Ctrl+C you will see `RuntimeError: Event loop is closed` and `ConnectionResetError` noise from Proactor teardown — cosmetic, not a leak, as long as `await engine.dispose()` runs in the lifespan shutdown. **Do not "fix" it** by switching to `WindowsSelectorEventLoopPolicy`: that caps you at 512 sockets and breaks asyncio subprocess support.
3. `--reload` **uses spawn, not fork.** The whole `app.main` module is re-imported in the child process, so nothing expensive or side-effecting may run at import time. Always pass the app as the import string `"app.main:app"`, never as an object. Any `run.py` you add needs an `if __name__ == "__main__":` guard.
4. **No Gunicorn on Windows.** For production, deploy in Docker or WSL2, or run `uvicorn --workers N` under NSSM as a Windows service. `--reload` and `--workers` are mutually exclusive.
5. `.env` **BOM** — see step 3.
6. **Skip** `python-magic`**.** It needs a `libmagic` DLL that is not present on Windows. Use `filetype` (pure Python, already in requirements), or sniff `%PDF-`, `\x89PNG` and `\xff\xd8\xff` yourself.
7. **Paths.** Use `pathlib.Path` everywhere and `tempfile.TemporaryDirectory()` — never a hardcoded `/tmp`. Mind the 260-character MAX_PATH limit: store the UUID as the filename and keep the user's original name only in the `original_filename` column.
8. **Console encoding** — `PYTHONUTF8=1`, see step 5.
9. **pytest-asyncio 1.x** removed the `event_loop` fixture — see step 8.
10. **Line endings** — `.gitattributes` before the first commit, see step 0.

Rebuilding this documentation

These PDFs are generated from the markdown in `docs/`. To regenerate after an edit:

```
cd C:\ocr
py -3.12 -m venv .docvenv
.\.docvenv\Scripts\python.exe -m pip install markdown pygments
.\.docvenv\Scripts\python.exe docs\build-pdf.py
```

Output lands in `docs\pdf\`: one PDF per section plus the combined `AP-Invoice-Automation-Blueprint.pdf`. Pass `--keep-html` to inspect the intermediate HTML in a browser, which is faster than reopening the PDF while iterating on content.

The script auto-detects Chrome, falling back to Edge. Both were present on this machine.